CRYPTO SCANNER

# Implementation Roadmap

Priorities for improving static crypto analysis of Go executables

**Goal**
Recover auditable algorithm evidence from an executable, connect crypto routines to application code where static analysis supports it, and preserve the limits of each finding.

## 1. Indirect-call and Go interface analysis

HIGHEST PRIORITY

Extend the direct-call graph to resolve indirect calls such as call rax, calls through memory addresses, Go function values, and interface method dispatch.

- Track candidate targets using register data flow and Go type/interface metadata.
- When a target cannot be resolved, record unresolved_indirect_call; do not invent a relationship.

Expected result: fewer missed relationships between application functions and crypto functions.

## 2. Improve Go symbol and inlining recovery

Go binaries can retain useful function and line information in symbol tables. Go’s debug/gosym package supports Go symbol and line tables.

- Handle stripped binaries and missing symbol records.
- Recover inline-function metadata where available.
- Support differences between Go compiler versions.

Expected result: better coverage when recognizable names are missing or crypto operations are inlined.

**Analysis rule**
A crypto function can be present or inlined without a simple direct call to a named symbol. Missing symbols must remain unknown rather than becoming guessed algorithm findings.

## 3. Expand the crypto signature database

Create a central rule registry for Go standard-library cryptography and third-party libraries. Add rules for AES, RSA, ECDSA, ECDH, hashes, HMAC, KDFs, AEAD modes, and post-quantum implementations such as ML-KEM and ML-DSA when their symbols can be identified.

Each rule should record:

- Symbol pattern
- Algorithm family, operation, variant, and aliases
- Required evidence level

Expected result: fewer missed algorithms and fewer incorrect mappings from generic names such as cipher, key, or crypto.

## 4. Algorithm-use classification and stronger evidence

Report one of these states for each algorithm:

| State | Meaning |
|---|---|
| **string_only** | A matching readable string was found. |
| **symbol_present** | A corresponding function symbol was identified. |
| **call_site_found** | A call instruction targets a recognized function. |
| **statically_reachable** | A recovered call path exists from an identified application root. |
| **runtime_observed** | Execution was independently observed during a test. |
| **unknown** | Available analysis cannot establish the state. |

Keep static reachability separate from runtime observation. A reachable function is not necessarily executed in every run.

## 5. Improve the CBOM export

- Deduplicate algorithm components while preserving evidence locations and call relationships.
- Include the executable's SHA-256, analyzer version, evidence type, function address, caller/callee, and analysis limitations.
- Validate against a supported CycloneDX schema and test import into Q-Risk.

CycloneDX CBOMs can represent algorithms, protocols, certificates, keys, tokens, and secrets, including relationships to software components.

## 6. Add a repeatable accuracy test suite

Build small known test executables and record expected scanner results:

| Fixture | What it checks |
|---|---|
| **AES use** | Known positive algorithm and call evidence. |
| **RSA use** | Known positive public-key evidence. |
| **String only: crypto/aes** | False-positive control; must not claim a linked function. |
| **Unreachable crypto code** | Separates symbol presence from static reachability. |
| **Indirect crypto call** | Checks function-pointer call resolution. |

Measure true positives, false positives, and missed findings. More findings alone do not prove better accuracy.

# Recommended implementation order

**Direct calls -> Indirect calls -> Call-graph reachability**
**-> Algorithm classification -> CBOM validation -> Accuracy testing**

> **Recommended next task**
> Implement indirect-call resolution in the existing crypto analysis and supporting disassembly/data-flow modules, with cases for register-indirect calls, memory-indirect calls, unresolved targets, and reachability. Keep the current project structure and extend its CBOM output.

# Q-Risk comparison: use the right success criterion

Do not require exact equality with a repository CBOM. A repository scan can include source code, tests, databases, and reports that cannot be reconstructed from the executable. Compare the algorithms represented in each artifact, retain evidence types for every match, and investigate disagreements.

> **Principle**
> A string match, linked symbol, static call path, and runtime execution are different kinds of evidence. Keep them distinct in reports and CBOM output.